

UniLearn

University Management System

Complete Technical Documentation

ASP.NET Core MVC • Entity Framework Core • SQL Server
Identity Framework • Role-Based Authorization

Information Technology Department — 4th Year | 2025/2026
Version 1.0

Table of Contents

- 1. Project Overview 3
- 2. System Architecture 4
- 3. Entity Relationship Diagram (ERD) 5
- 4. Database Schema 6
- 5. Roles & Authorization 8
- 6. Authentication & Authorization Flow 9
- 7. Module Workflows 10
- 8. Controllers & Actions Reference 12
- 9. Views Reference 14
- 10. Technology Stack 15
- 11. Security Considerations 16
- 12. Deployment & Configuration 17

■ 1. Project Overview

Project Summary

UniLearn is a full-featured **University Management System** built with **ASP.NET Core MVC (.NET 8)**. It provides a Udemy-inspired web interface for managing academic entities — Departments, Instructors, Courses, Trainees, and Student Enrollments — with **role-based access control** differentiating between Admins and Students.

Key Objectives

- Centralize management of academic data in a single web application.
- Provide role-based access: Admins can perform full CRUD; Students can browse and enroll.
- Enable students to track their enrolled courses and grades through a personal profile.
- Maintain referential integrity across Departments, Courses, Instructors, and Trainees.
- Deliver a responsive, modern UI using Bootstrap 5 and Font Awesome icons.

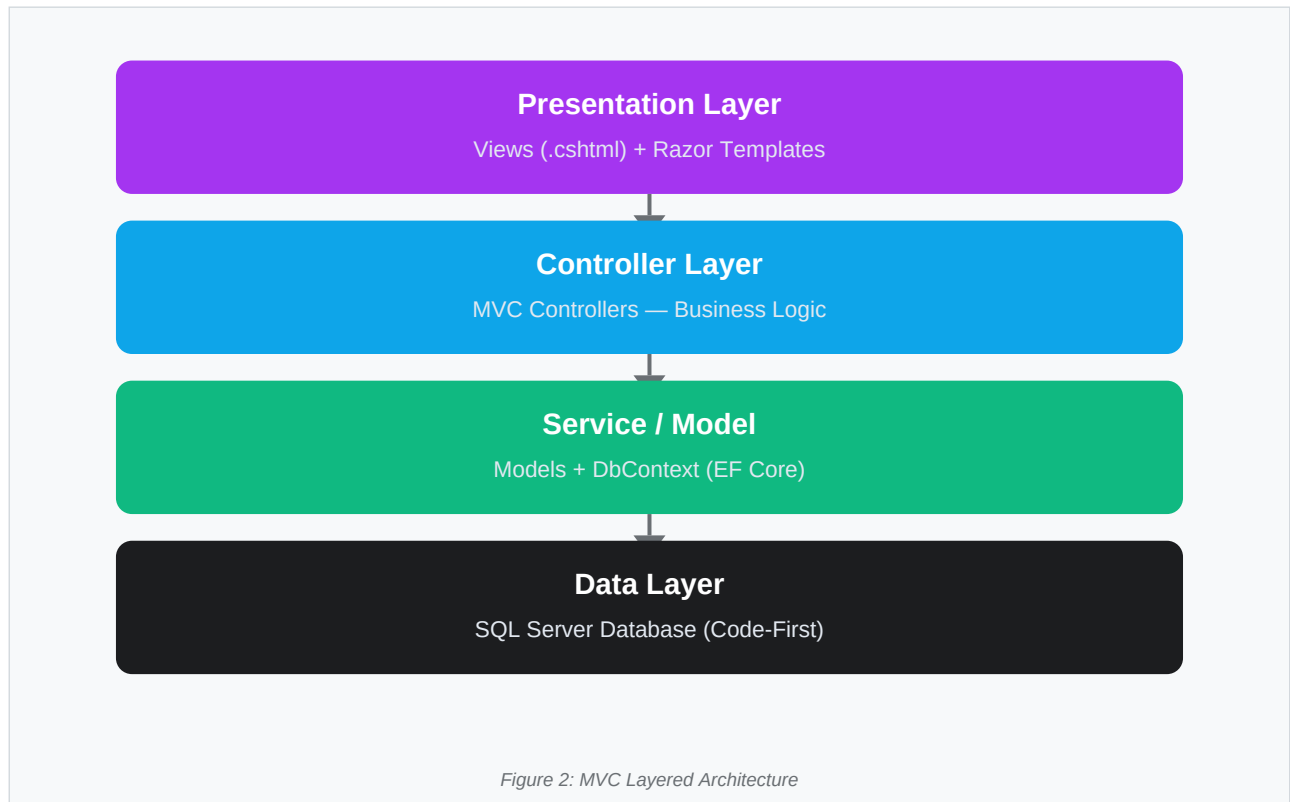
Core Modules

Module	Description	Access
Authentication	Login, Register, Logout via ASP.NET Identity	All users
Departments	CRUD for academic departments; lists instructors, courses, trainees	View: All CRUD: Admin
Instructors	CRUD + photo upload + search by name	View: All CRUD: Admin
Courses	CRUD + enrollment management + hour tracking	View: All CRUD: Admin
Trainees	CRUD + photo upload + course results	Admin only
Enrollment	Enroll/unenroll from courses; view personal course list	Authenticated users
Profile	Personal dashboard showing enrolled courses and grades	Authenticated users
Admin Dashboard	High-level stats and management links	Admin only

■ 2. System Architecture

MVC Layered Architecture

UniLearn follows the classic **Model-View-Controller (MVC)** pattern as enforced by ASP.NET Core. Each layer has a distinct responsibility, enabling clean separation of concerns and maintainability.

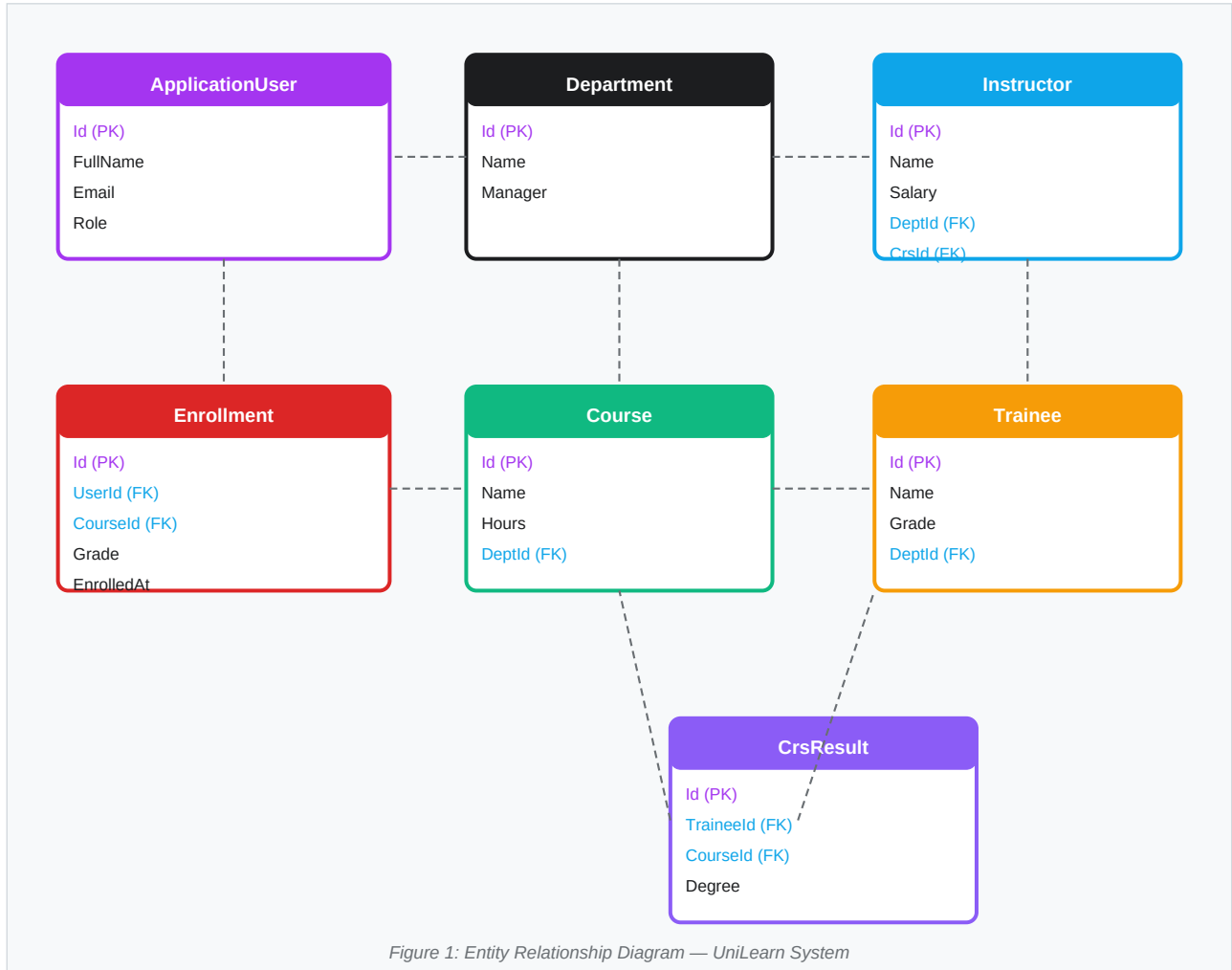


Layer	Components	Responsibility
Presentation	Views/*.cshtml, _Layout.cshtml, Razor Tag Helpers	Render HTML, handle user input forms, display data
Controller	DepartmentController, InstructorController, CourseController, TraineeController, EnrollmentController	Receive HTTP requests, control flow, coordinate data
Model	Department, Instructor, Course, Trainee, Enrollment	Represent data entities, contain data annotations for validation
Data Access	AppDbContext (EF Core), SQL Server Migrations	ORM queries, relationship mapping, schema migration
Infrastructure	ASP.NET Identity, appsettings.json, wwwroot	Auth, configuration, static assets (CSS, JS, images)

■ 3. Entity Relationship Diagram (ERD)

Data Model Overview

The system contains **7 core entities**. The diagram below illustrates their relationships. Foreign keys enforce referential integrity at the database level via Entity Framework Core Code-First migrations.



Entity Relationships Summary

Relationship	Cardinality	Description
Department → Instructor	1 : Many	A department employs many instructors
Department → Course	1 : Many	A department offers many courses
Department → Trainee	1 : Many	A department has many trainees
Instructor → Course	1 : 0..1	An instructor may teach one course (optional)
Course → Enrollment	1 : Many	A course can have many student enrollments
ApplicationUser → Enrollment	1 : Many	A user (student) can enroll in many courses

Relationship	Cardinality	Description
Trainee → CrsResult	1 : Many	A trainee has results for multiple courses
Course → CrsResult	1 : Many	A course has results for multiple trainees

■ 4. Database Schema

Table: Departments

Column	Type	Constraints	Description
Id	INT	PK, Identity	Auto-increment primary key
Name	NVARCHAR(MAX)	NOT NULL	Department name
Manager	NVARCHAR(MAX)	NULL	Manager full name (text)

Table: Instructors

Column	Type	Constraints	Description
Id	INT	PK, Identity	Auto-increment primary key
Name	NVARCHAR(MAX)	NOT NULL	Instructor full name
Address	NVARCHAR(MAX)	NULL	Home or office address
Salary	DECIMAL(18,2)	NOT NULL	Monthly salary
Image	NVARCHAR(MAX)	NULL	Relative path to uploaded photo
DepartmentId	INT	FK → Departments.Id	Owning department
CourseId	INT	FK → Courses.Id, NULL	Assigned course (optional)

Table: Courses

Column	Type	Constraints	Description
Id	INT	PK, Identity	Auto-increment primary key
Name	NVARCHAR(MAX)	NOT NULL	Course title
Hours	INT	NOT NULL	Credit hours
DepartmentId	INT	FK → Departments.Id	Owning department

Table: Trainees

Column	Type	Constraints	Description
Id	INT	PK, Identity	Auto-increment primary key
Name	NVARCHAR(MAX)	NOT NULL	Trainee full name
Address	NVARCHAR(MAX)	NULL	Home address
Grade	NVARCHAR(MAX)	NULL	Academic grade/year

Column	Type	Constraints	Description
Image	NVARCHAR(MAX)	NULL	Relative path to photo
DepartmentId	INT	FK → Departments.Id	Enrolled department

Table: Enrollments

Column	Type	Constraints	Description
Id	INT	PK, Identity	Auto-increment primary key
UserId	NVARCHAR	FK → AspNetUsers.Id	Enrolled student (Identity user)
CourseId	INT	FK → Courses.Id	Enrolled course
Grade	DECIMAL	NULL	Grade awarded after completion
EnrolledAt	DATETIME	NOT NULL, DEFAULT GETDATE()	Enrollment timestamp

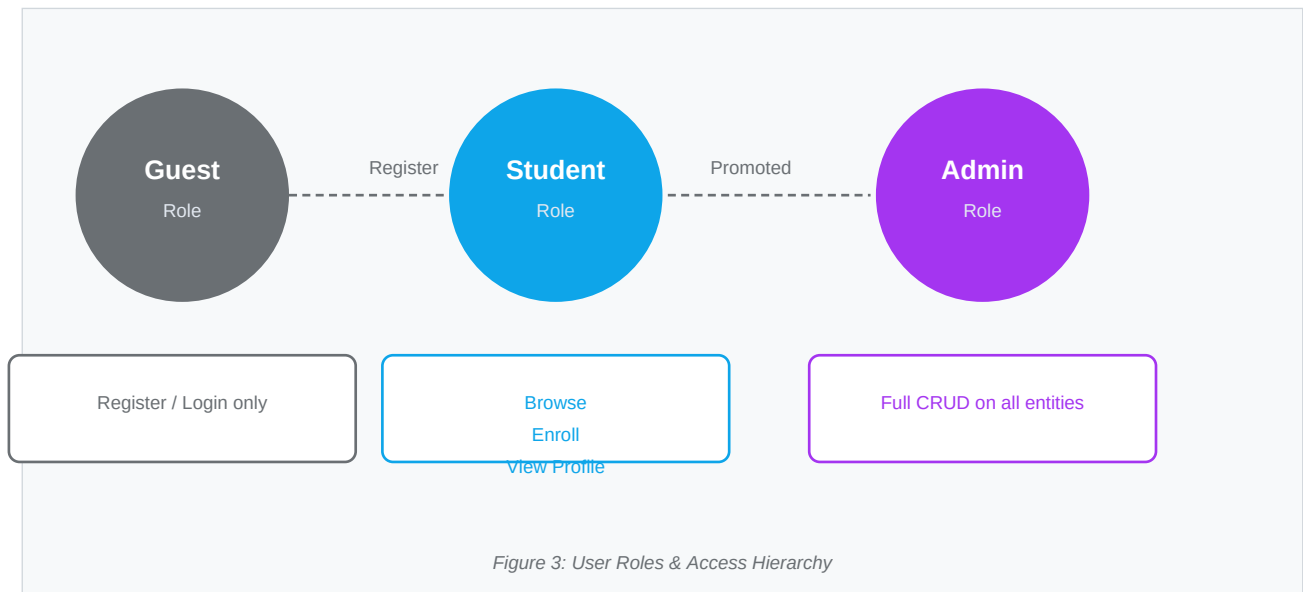
Table: CrsResults

Column	Type	Constraints	Description
Id	INT	PK, Identity	Auto-increment primary key
TraineeId	INT	FK → Trainees.Id	The trainee
CourseId	INT	FK → Courses.Id	The course
Degree	INT	NOT NULL	Numeric score/degree

ASP.NET Identity Tables (auto-generated)

ASP.NET Core Identity automatically creates and manages the following tables: **AspNetUsers** (extended by ApplicationUser with FullName), **AspNetRoles**, **AspNetUserRoles**, **AspNetUserClaims**, **AspNetRoleClaims**, **AspNetUserLogins**, **AspNetUserTokens**.

■ 5. Roles & Authorization

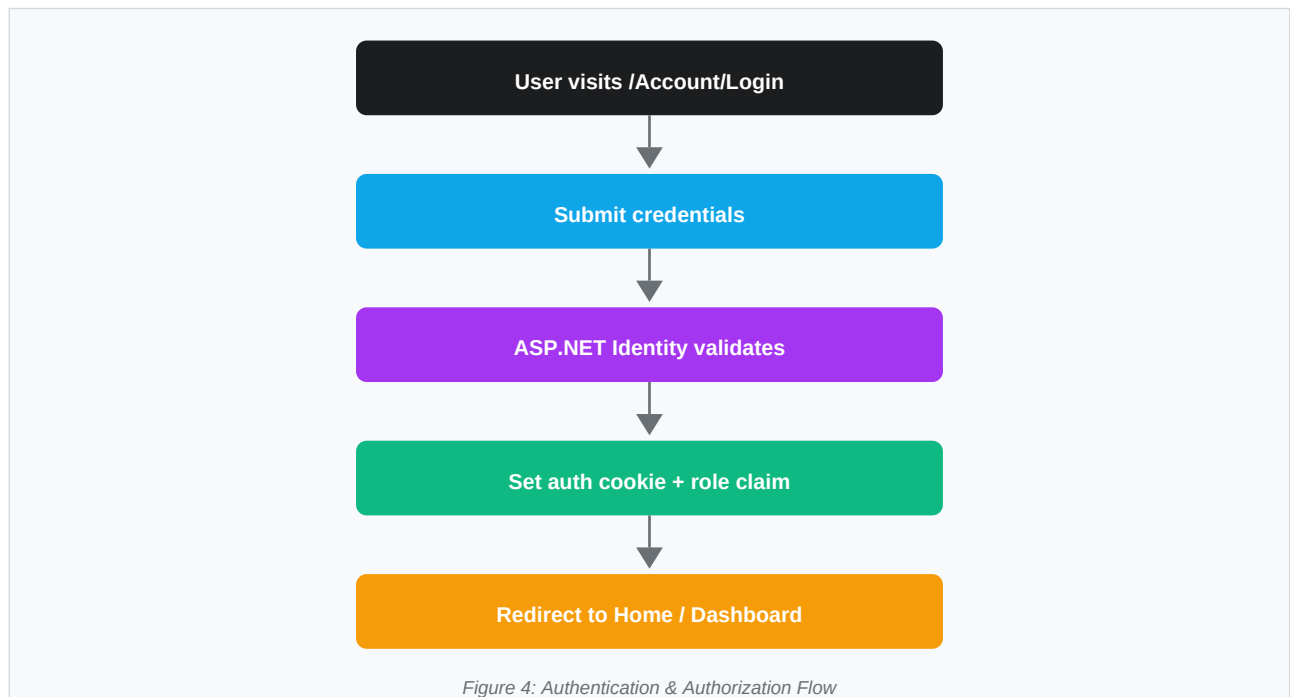


Feature / Action	Guest (Unauthenticated)	Student (Authenticated)	Admin
View Home Page	✓	✓	✓
View Instructors List	✓	✓	✓
View Instructor Detail	✓	✓	✓
View Salary / Address	✗	✗	✓
Add / Edit / Delete Instructor	✗	✗	✓
View Departments	✓	✓	✓
Add / Edit / Delete Department	✗	✗	✓
View Courses	✓	✓	✓
Enroll in a Course	✗	✓	✓
View My Courses / Profile	✗	✓	✓
View Trainees	✗	✗	✓
Add / Edit / Delete Trainee	✗	✗	✓
Admin Dashboard	✗	✗	✓
Manage All Enrollments	✗	✗	✓

■ 6. Authentication & Authorization Flow

Login & Session Management

The application uses **ASP.NET Core Identity** with cookie-based authentication. Upon successful login, an encrypted cookie is issued containing the user's claims and roles. Every controller action decorated with `[Authorize]` or `[Authorize(Roles="Admin")]` verifies this cookie automatically.

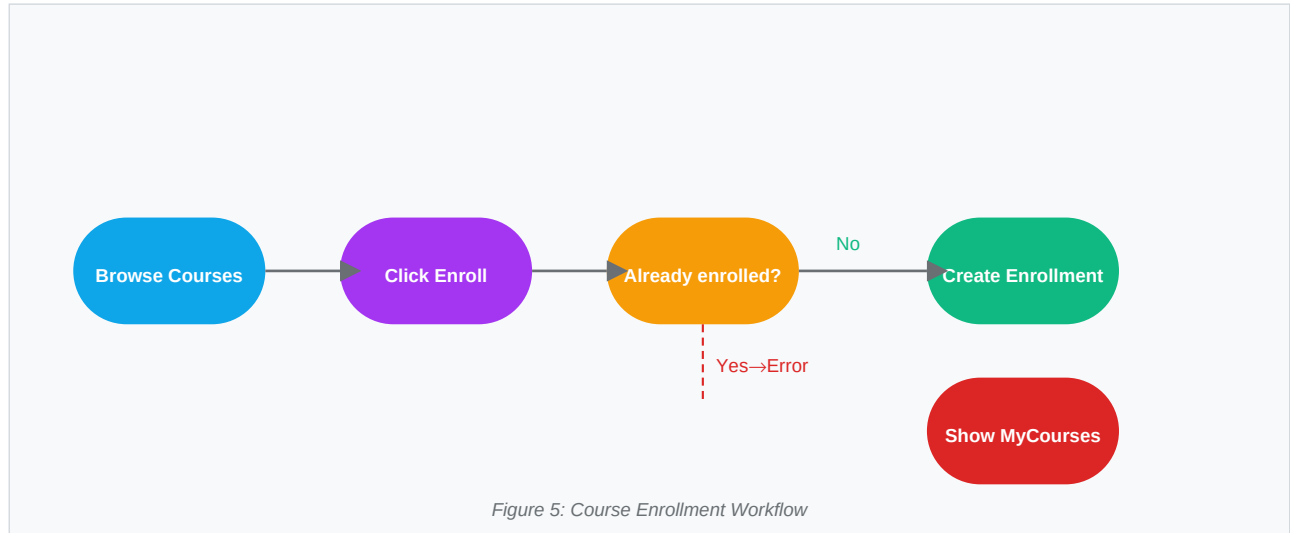


AccountController Actions

Action	HTTP	Route	Description
Login (GET)	GET	/Account/Login	Display login form
Login (POST)	POST	/Account/Login	Validate credentials; sign in; redirect
Register (GET)	GET	/Account/Register	Display registration form
Register (POST)	POST	/Account/Register	Create Identity user; assign Student role; sign in
Logout (POST)	POST	/Account/Logout	Sign out; invalidate cookie; redirect to /

■ 7. Module Workflows

7.1 Course Enrollment Workflow



When a student clicks **Enroll** on a course detail page, the EnrollmentController checks for an existing enrollment record. If one exists, it returns an error/alert. Otherwise it creates a new Enrollment row with the current UserId, CourseId, and timestamp, then redirects to **My Courses**.

7.2 Instructor CRUD Workflow

Step	Actor	Action	Result
1	Admin	Navigate to /Instructor	List of all instructors shown
2	Admin	Click Add Instructor → /Instructor/Create	Form shown with Dept & Course dropdowns
3	Admin	Fill form + upload photo → Submit	POST to Create; image saved to wwwroot/images
4	System	Save Instructor entity to DB	Redirect to /Instructor index
5	Admin	Click Edit icon → /Instructor/Edit/{id}	Pre-filled edit form rendered
6	Admin	Modify fields → Submit	POST to Edit; DB updated; redirect
7	Admin	Click Delete → /Instructor/Delete/{id}	Confirmation page shown
8	Admin	Confirm Delete → POST	Record removed; redirect to index

7.3 Department Detail Workflow

The Department Detail page (**/Department/Details/{id}**) performs eager loading via EF Core Include() to fetch related **Instructors**, **Courses**, and **Trainees** in a single query, then renders them in three side-by-side cards using Bootstrap grid. Counts are displayed in badge elements.

7.4 Image Upload Workflow

Step	Description
1 — Form	enctype='multipart/form-data' enables binary file transfer in the POST
2 — Controller	IFormFile imageFile parameter receives the upload
3 — Validate	Check file is not null and has valid content type
4 — Save	Generate unique filename; copy stream to wwwroot/images/
5 — Store path	Save relative URL (e.g. /images/abc123.jpg) to the Model.Image property
6 — Display	Razor view renders or initials avatar fallback

■ 8. Controllers & Actions Reference

DepartmentController

Action	Method	Route	Auth	Description
Index	GET	/Department	Public	List all departments with counts
Details	GET	/Department/Details/{id}	Public	Show dept + instructors/courses/trainee
Create	GET	/Department/Create	Admin	Show create form
Create	POST	/Department/Create	Admin	Save new department
Edit	GET	/Department/Edit/{id}	Admin	Show pre-filled edit form
Edit	POST	/Department/Edit/{id}	Admin	Update department record
Delete	GET	/Department/Delete/{id}	Admin	Confirm delete page
Delete	POST	/Department/Delete/{id}	Admin	Remove department

InstructorController

Action	Method	Route	Auth	Description
Index	GET	/Instructor?searchName=	Public	List instructors; optional name search
Detail	GET	/Instructor/Detail/{id}	Public	Show instructor profile
Create	GET	/Instructor/Create	Admin	Show create form + dept/course dropd
Create	POST	/Instructor/Create	Admin	Save; handle image upload
Edit	GET	/Instructor/Edit/{id}	Admin	Pre-fill edit form
Edit	POST	/Instructor/Edit/{id}	Admin	Update; replace image if new one uplo
Delete	GET	/Instructor/Delete/{id}	Admin	Confirm delete
Delete	POST	/Instructor/Delete/{id}	Admin	Remove instructor record

EnrollmentController

Action	Method	Route	Auth	Description
MyCourses	GET	/Enrollment/MyCourses	User	Show logged-in user's enrollments
Enroll	POST	/Enrollment/Enroll	User	Create enrollment; prevent duplicates
Unenroll	POST	/Enrollment/Unenroll	User	Delete enrollment record
Profile	GET	/Enrollment/Profile	User	Show user profile + enrollment table

TraineeController

Action	Method	Route	Auth	Description
Index	GET	/Trainee	Admin	List all trainees
Detail	GET	/Trainee/Detail/{id}	Admin	Show trainee + CrsResults
Create	GET/POST	/Trainee/Create	Admin	Add new trainee with photo
Edit	GET/POST	/Trainee/Edit/{id}	Admin	Edit trainee info
Delete	GET/POST	/Trainee/Delete/{id}	Admin	Remove trainee

■ 9. Views Reference

View File	Module	Model Type	Key Features
_Layout.cshtml	Shared	—	Navbar, footer, Bootstrap 5, Font Awesome
Department/Index.cshtml	Department	IEnumerable<Department>	Table with counts; Admin-only Add button
Department/Details.cshtml	Department	Department	3-column card layout: instructors, courses
Department/Edit.cshtml	Department	Department	Simple 2-field form; validation spans
Instructor/Index.cshtml	Instructor	IEnumerable<Instructor>	Search bar; avatar/initials; Admin salary
Instructor/Detail.cshtml	Instructor	Instructor	Profile card; conditional salary (Admin only)
Instructor/Create.cshtml	Instructor	Instructor	DropDownList for Dept & Course; file upload
Instructor/Edit.cshtml	Instructor	Instructor	Hidden image field; new photo upload
Instructor/Delete.cshtml	Instructor	Instructor	Confirmation card with red icon
Trainee/Index.cshtml	Trainee	IEnumerable<Trainee>	Admin-only access guard at top
Trainee/Detail.cshtml	Trainee	Trainee	Shows CrsResults list with degree badge
Enrollment/MyCourses.cshtml	Enrollment	IEnumerable<Enrollment>	Card grid; unenroll form per card
Enrollment/Profile.cshtml	Enrollment	ViewBag (dynamic)	User info + enrollment table + grade book
Home/Index.cshtml	Home	—	Hero banner; 4 stat cards
Account/Login.cshtml	Account	LoginViewModel	Email + password form
Account/Register.cshtml	Account	RegisterViewModel	FullName + email + password registration
Shared/Error.cshtml	Shared	ErrorViewModel	Request ID display; dev mode hint

■ 10. Technology Stack

Technology	Version / Detail	Purpose
ASP.NET Core MVC	.NET 8	Web framework; routing, controllers, Razor views
Entity Framework Core	8.x Code-First	ORM; database migrations; LINQ queries
SQL Server / LocalDB	2019+	Relational database backend
ASP.NET Core Identity	Built-in	User management; password hashing; cookie auth
Bootstrap	5.3.0 (CDN)	Responsive CSS grid; components; utilities
Font Awesome	6.4.0 (CDN)	Icon library used throughout the UI
Google Fonts — Inter	Variable weight	Primary sans-serif typeface
jQuery Validation	bundled in lib/	Client-side form validation (unobtrusive)
Razor Tag Helpers	asp-for, asp-action, asp-route-*	Server-side HTML generation
C#	12 / .NET 8	Primary backend language

Project Folder Structure

Path	Description
Controllers/	All MVC controllers (one per module)
Models/	Entity classes + ViewModels + DbContext
Views/	Razor .cshtml files organized by controller
Views/Shared/_Layout.cshtml	Master layout with navbar and footer
wwwroot/	Static assets: CSS, JS, uploaded images
wwwroot/images/	User-uploaded instructor & trainee photos
Migrations/	EF Core auto-generated migration files
appsettings.json	Connection string, Identity config, logging
Program.cs	App bootstrap: DI registration, middleware pipeline

■ 11. Security Considerations

■ Anti-Forgery Tokens

Every POST form includes `@Html.AntiForgeryToken()`. The `[ValidateAntiForgeryToken]` attribute on POST actions prevents CSRF attacks.

■ Role-Based Authorization

Controllers and Razor views use `[Authorize(Roles="Admin")]` and `User.IsInRole("Admin")` respectively to guard sensitive operations.

■ Password Hashing

ASP.NET Identity uses PBKDF2 with HMAC-SHA256 and a per-user salt to store passwords. Plain-text passwords are never persisted.

■ Input Validation

Data Annotations on model properties (e.g. `[Required]`, `[MaxLength]`) provide server-side validation. Client-side jQuery Unobtrusive Validation provides immediate feedback.

■ File Upload Safety

Uploaded images should be validated for MIME type and size. Saved files are stored outside the application root or under a non-executable folder.

■ SQL Injection Prevention

Entity Framework Core uses parameterized queries exclusively — no raw SQL interpolation is present in the codebase.

■ Sensitive Data Hiding

Instructor salary and address are conditionally rendered only for Admin users, both in list and detail views.

■ HTTPS

ASP.NET Core is configured to redirect HTTP to HTTPS by default via `UseHttpsRedirection()` middleware.

■ 12. Deployment & Configuration

Prerequisites

- .NET 8 SDK / Runtime installed
- SQL Server or SQL Server Express / LocalDB
- Visual Studio 2022 or VS Code with C# extension

Setup Steps

Step	Command / Action
1. Clone repo	git clone <repository-url> && cd UniversityApp_1
2. Configure DB	Edit appsettings.json → ConnectionStrings → DefaultConnection
3. Apply migrations	dotnet ef database update
4. Seed Admin role	Run app; manually assign Admin role via AspNetUserRoles or seed code
5. Run application	dotnet run OR press F5 in Visual Studio
6. Access in browser	https://localhost:5001 (port may vary)

appsettings.json Key Settings

The connection string key is **DefaultConnection**. For local development, LocalDB is used: `Server=(localdb)\mssqllocaldb;Database=UniversityDb;Trusted_Connection=True;`. For production, replace with a full SQL Server connection string and set the environment to Production.

Static File Configuration

Uploaded images are served via `UseStaticFiles()` middleware from **wwwroot/**. Ensure the web host process has **write permission** to `wwwroot/images/`. For Azure App Service, consider using Azure Blob Storage for persistent image storage.

■ Project Summary Card

Attribute	Value
Application Name	UniLearn — University Management System
Framework	ASP.NET Core MVC (.NET 8)
Database	SQL Server with EF Core Code-First
Authentication	ASP.NET Identity (Cookie-based)
Roles	Admin, Student (default)
Entities	Department, Instructor, Course, Trainee, Enrollment, CrsResult, ApplicationUs
Controllers	7 (Account, Admin, Department, Instructor, Course, Trainee, Enrollment)
Views	~18 Razor .cshtml files
UI Framework	Bootstrap 5.3 + Font Awesome 6.4
Image Storage	wwwroot/images/ (local file system)
Academic Year	2025 / 2026 — 4th Year IT